

INVENTION DISCLOSURE FORM

Date of Submission of Form for patent processing :

Client Billing Information

Contact Person :

Address :

Telephone/ mobile :

Fax :

E-mail :

Patent Applicant Details

Full Name :

QpiAI

Address :

Mezzanine Floor (Northwest and Southwest Bays), Hub 1 SEZ Tower, Karle Town Centre, Karle Infra
Special Economic Zone, Nagavara, Bengaluru, Karnataka 560045

Nationality :

Telephone/ mobile :

Fax :

E-mail :

Inventor(s) Details

(Please list the inventors in the order their names will appear on the patent.)

Name :

Citizenship :

Mailing Address :

Contact Number :

E-mail :

Name :

Citizenship :

Mailing Address :

Contact Number :

E-mail :

Name :

Citizenship :

Mailing Address :

Contact Number :

E-mail :

Name :

Citizenship :

Mailing Address :

Contact Number :

E-mail :

INVENTION DISCLOSURE

Provide a Descriptive Title of Invention.

METHOD AND SYSTEM FOR MEMORY-ADAPTIVE LAYER-SHARDED INFERENCE AND TRAINING OF LARGE LANGUAGE MODELS ACROSS HETEROGENEOUS HARDWARE

Provide a list of relevant known prior art documents (patents, published articles, etc.). Please only indicate documents already known to the inventors. A separate search need not be done to fill in this field.

Layer-Sharding and Memory-Offloading Inference:

- Gavin Li (lyogavin). "AirLLM: run 70B LLM inference on a single 4GB GPU via layer-by-layer loading." GitHub / HuggingFace (2023). FOUNDATIONAL BASE — loads exactly one layer at a time.
- Liu, Y., et al. "PIPO: Pipelined Offloading for Efficient Inference on Consumer Devices." arXiv:2504.03664 (2025).
- Sheng, Y., et al. "FlexGen: High-Throughput Generative Inference of LLMs with a Single GPU." ICML 2023, arXiv:2303.06865.
- Alizadeh, K., et al. "LLM in a Flash: Efficient LLM Inference with Limited Memory." ACL 2024, arXiv:2312.11514.

Speculative Decoding:

- Qualcomm Inc. "Efficient Speculative Decoding in Autoregressive Generative AI Models." US Patent Application US20250245430A1 (priority 2023).
- Svirschevski, R., et al. "SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices." NeurIPS 2024, arXiv:2406.02532.
- Huang, K., et al. "SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths." COLM 2025, arXiv:2405.19715.

On-Device / Single-GPU Training and Offloading:

- Tang, Y., et al. "Fuyou: Adding NVMe SSDs to Enable and Accelerate 100B Model Fine-tuning on a Single GPU." arXiv:2403.06504 (2024).
- Ren, J., et al. "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning." SC21, arXiv:2104.07857.
- Rajbhandari, S., et al. "ZeRO-Offload: Democratizing Billion-Scale Model Training." USENIX ATC 2021.
- Dettmers, T., et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS 2023, arXiv:2305.14314.
- Hu, E., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022, arXiv:2106.09685.

Heterogeneous CPU+GPU Execution:

- Kamahori, K., et al. "Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models." ICLR 2025, arXiv:2402.07033.
- Song, X., et al. "SpecOffload: Unlocking Latent GPU Capacity for LLM Inference." arXiv:2505.10259 (2025).
- Gerganov, G. "llama.cpp: LLM inference in C/C++." GitHub (2023).

Related QpiAI Application:

- QpiAI. “Method and System for Hardware-Aware Deployment and Optimization of Large Language Models across Heterogeneous Computing Architectures.” Application No. 144678-0132 (2025).
Complementary — see relationship note below.

Provide a brief summary of the state of the art and its limitations. Also indicate what problem(s) does the Invention solve?

Current State of Large Language Model Execution on Constrained Hardware

Large language models (LLMs) such as the LLaMA, Qwen, Mistral, Gemma, Phi, and DeepSeek families now range from one billion to over one trillion parameters. Executing these models — for either inference or training — on widely available consumer and edge hardware is fundamentally constrained by memory. A 70-billion-parameter model in 16-bit precision requires approximately 140 gigabytes of GPU memory for inference alone, and full fine-tuning requires well over one terabyte. The overwhelming majority of available computing devices — laptops, desktops, edge boxes, and entry-level GPUs — have between 4 and 24 gigabytes of accelerator memory, or none at all. Existing approaches address this gap only partially:

1. Fixed, Non-Adaptive Layer Loading

The foundational layer-sharding approach (AirLLM) makes 70B inference possible on a 4 GB GPU by loading one transformer layer at a time from storage. However, it loads exactly one layer per read regardless of how much accelerator memory is actually free. On a device with spare memory, the available capacity is wasted; on a device with less memory, the system has no graceful fallback. The loading policy is hardcoded and does not change as generation proceeds or as the key-value cache grows.

2. No On-Device Training of Large Models

Existing parameter-efficient fine-tuning methods (such as QLoRA) require the entire quantized model to reside in GPU memory simultaneously, which is infeasible for 70B-class models on 4–8 GB devices. Server-class offloading systems (ZeRO-Offload, ZeRO-Infinity) require multiple GPUs connected by collective-communication networks and large server memory. There is no system that performs full backward-pass training of a 70B-class model on a single consumer device by streaming weights from storage.

3. Static Speculative Decoding

Speculative decoding accelerates generation by drafting several candidate tokens with a small model and verifying them with the large model. Existing systems fix the number of drafted tokens, or require training an auxiliary prediction component to vary it. None exploits the specific property of storage-streamed inference that verifying many tokens costs essentially the same as verifying one.

4. Static CPU/GPU Partitioning

Recent heterogeneous-execution systems split work between CPU and GPU using a fixed assignment decided once (for example, all attention on CPU and all feed-forward on GPU, or routing only mixture-

of-experts weights to CPU). None re-decides the placement dynamically as memory conditions change during generation.

Organising Principle — Two Levers

All mechanisms of the invention reduce to two levers. Lever one is to load fewer bytes from storage per generated token; lever two is to extract more useful work from each byte that is loaded. The same three memory tiers are used as in the baseline, but each byte that crosses the storage and interconnect links is made to do far more work. Lever one is realised by sparsity / mixture-of-experts routing (loading only the experts a token actually uses), per-layer mixed precision, and deterministic prefetch that exploits the known, fixed forward execution order to replace cache-eviction heuristics with a scheduled ring buffer. Lever two is realised by cross-request batching (loading a shard once and pushing many queued sequences through it before eviction), tree-structured speculative verification (checking a whole tree of candidate tokens in a single constant-cost sweep), and idle-tier overlap (using the storage and interconnect bandwidth that sits idle during the draft phase to prefetch the verifier's shards).

Problem Solved by the Invention

The disclosed invention provides a single execution engine that runs both inference and training of LLMs far larger than available memory, on essentially any hardware, by adapting its memory strategy in real time. It (1) measures available memory after every generated token and groups as many layers as will fit into each storage read; (2) performs full on-device training through a reverse-order streaming backward pass with a disk-resident optimizer, requiring no minimum accelerator memory and no multi-GPU communication; (3) adds a training-free dynamic speculative decoder that adjusts its draft length automatically; and (4) dynamically assigns each group of layers to the CPU or the accelerator per token based on current memory availability — degrading gracefully all the way down to CPU-only execution when no accelerator memory remains. The system has been reduced to practice for inference and QLoRA training of 7-billion- and 70-billion-parameter LLaMA-family models on a single computing device having 4 gigabytes of GPU memory and 16 gigabytes of CPU memory.

Please provide representative figures/diagrams of the Invention.

Nine figures are provided below (FIG. 1 – FIG. 9), each followed by a brief description.

FIT LLM Architecture — Three-Tier Memory Hierarchy

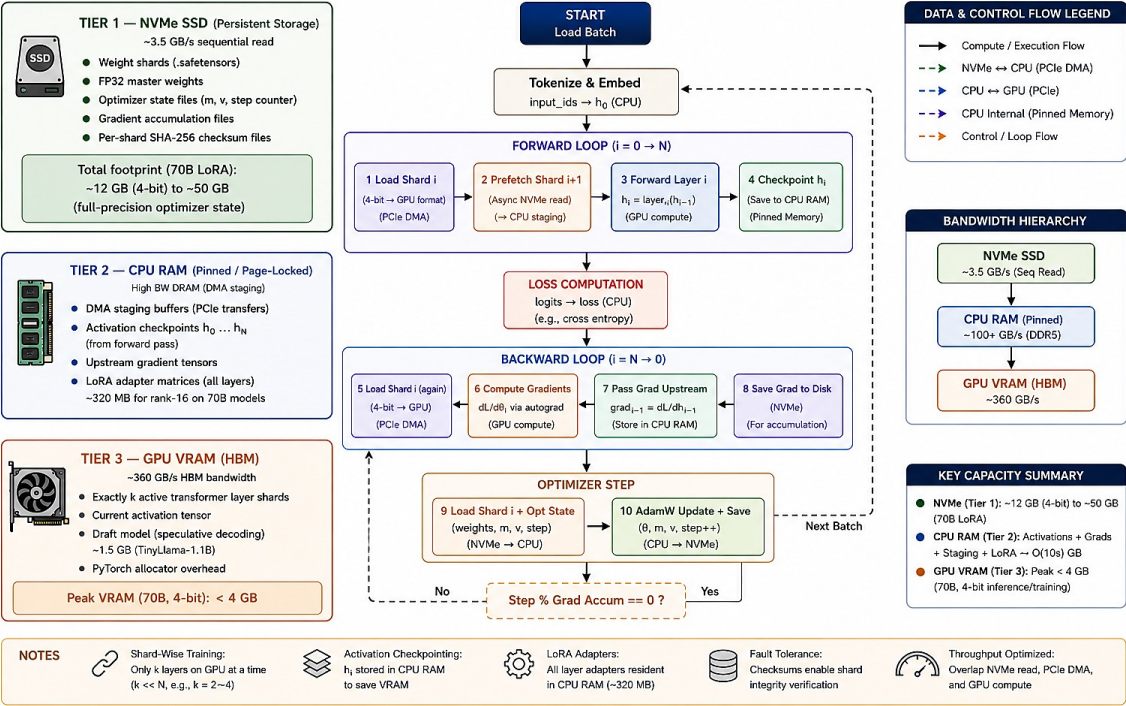
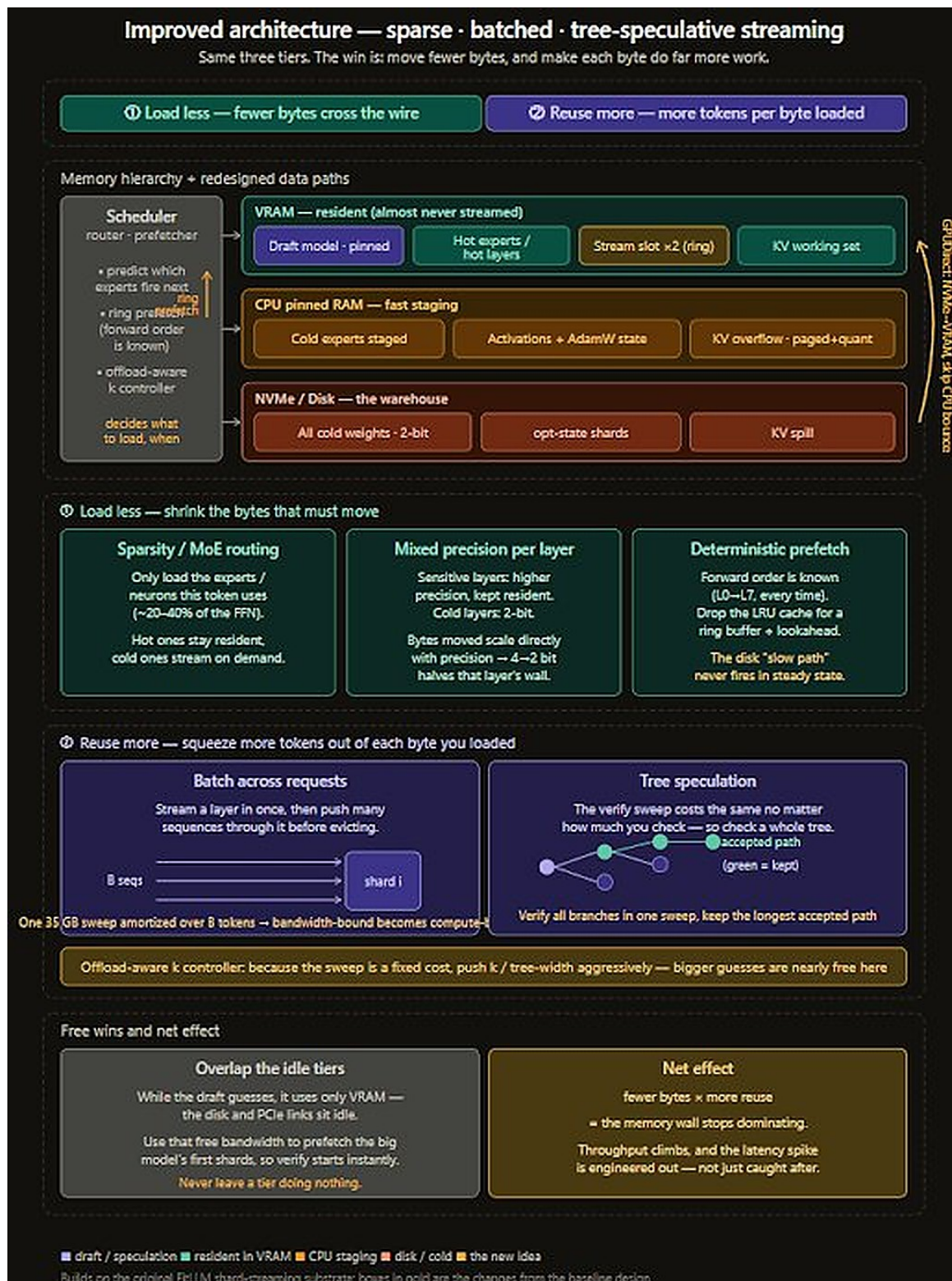


FIG. 1. Full system architecture across a three-tier memory hierarchy (NVMe storage, CPU RAM, accelerator memory), showing the end-to-end forward pass, backward pass, and optimizer step, the data and control flow between tiers, and the relative bandwidth of each tier.



idle-tier overlap prefetch. A scheduler / router-prefetcher places hot data in accelerator memory, warm data staged in CPU pinned RAM, and cold weights on storage.

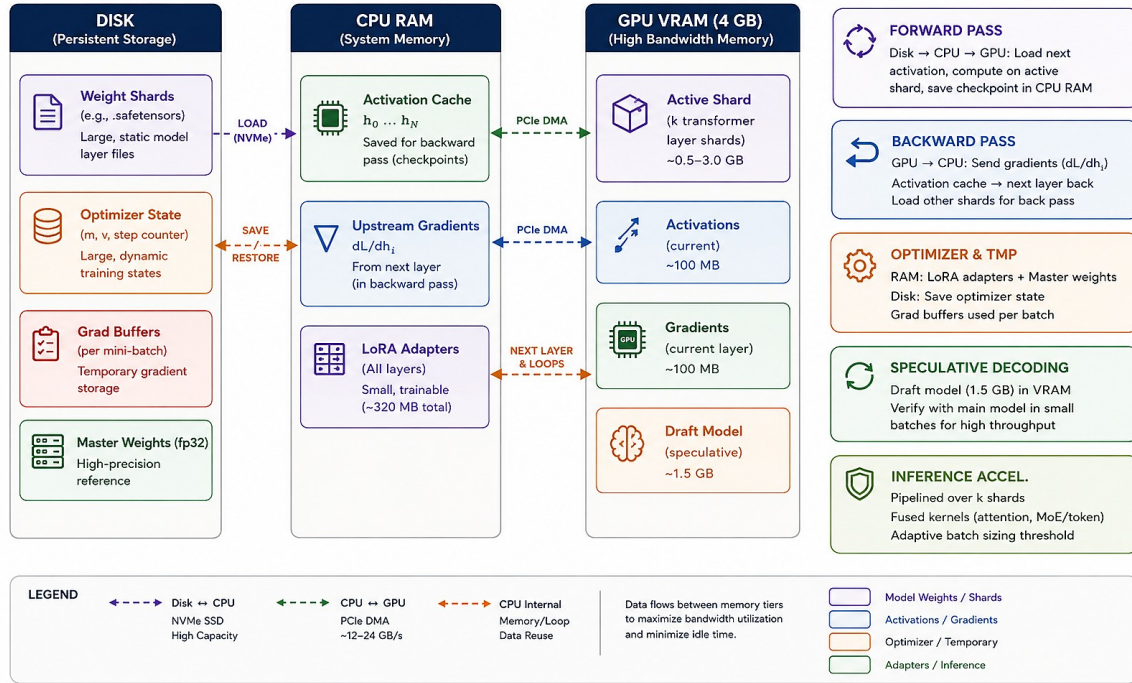


FIG. 3. Memory-tier contents and data movement: which tensors reside in storage, CPU RAM, and accelerator memory during forward and backward passes, and the transfers between them.

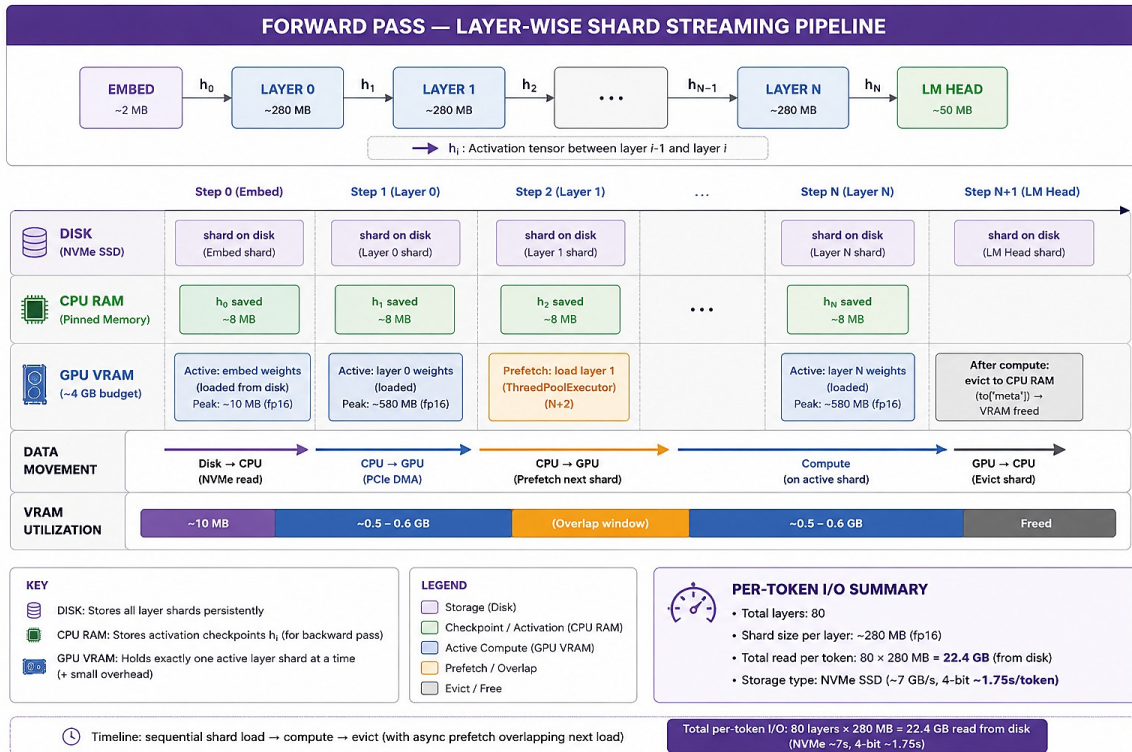


FIG. 4. Forward pass — layer-wise shard-streaming pipeline: shards are loaded, computed, and evicted in sequence while the next shard is prefetched, with activation checkpoints saved to CPU RAM for the backward pass.

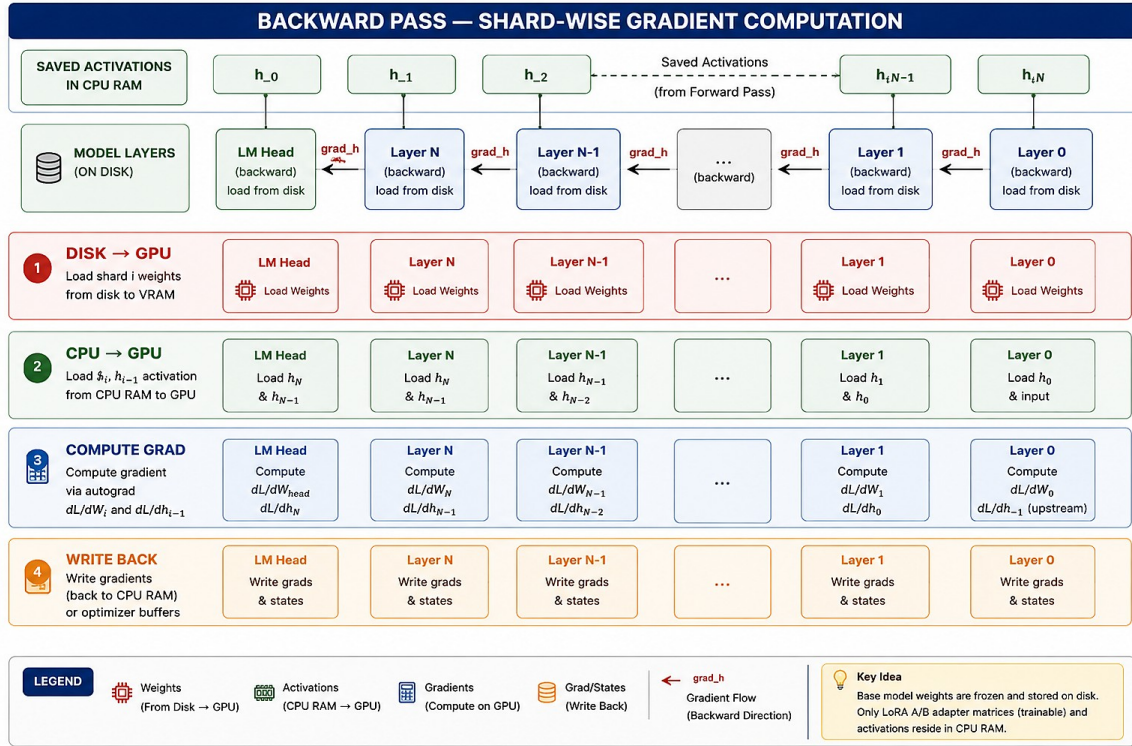


FIG. 5. Backward pass — shard-wise reverse-order gradient computation: weight shards are streamed from storage in reverse layer order and the saved activation checkpoints are reloaded from CPU RAM to reconstruct the local computation graph one shard at a time.

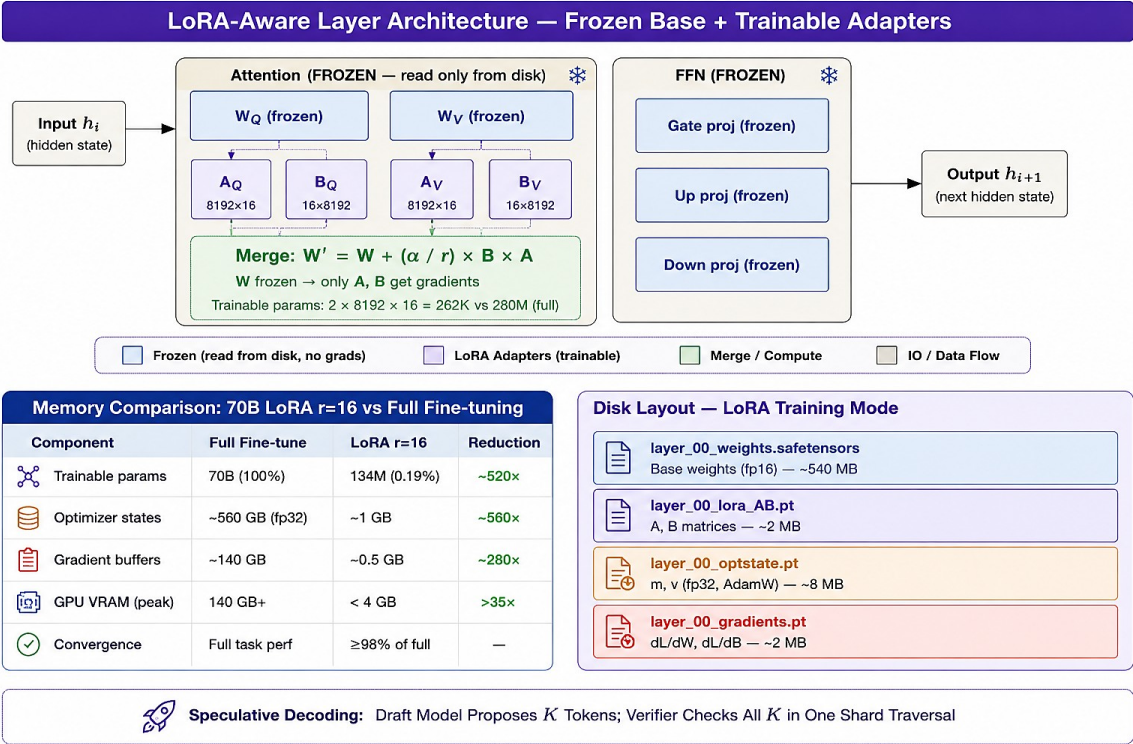


FIG. 6. Low-rank adapter (LoRA) layer architecture: the base weights are frozen and read from storage; only the small adapter matrices are trainable and held in CPU RAM, with a comparison of trainable parameters and memory against full fine-tuning.

Algorithm 2 — Inference (speculative decoding)

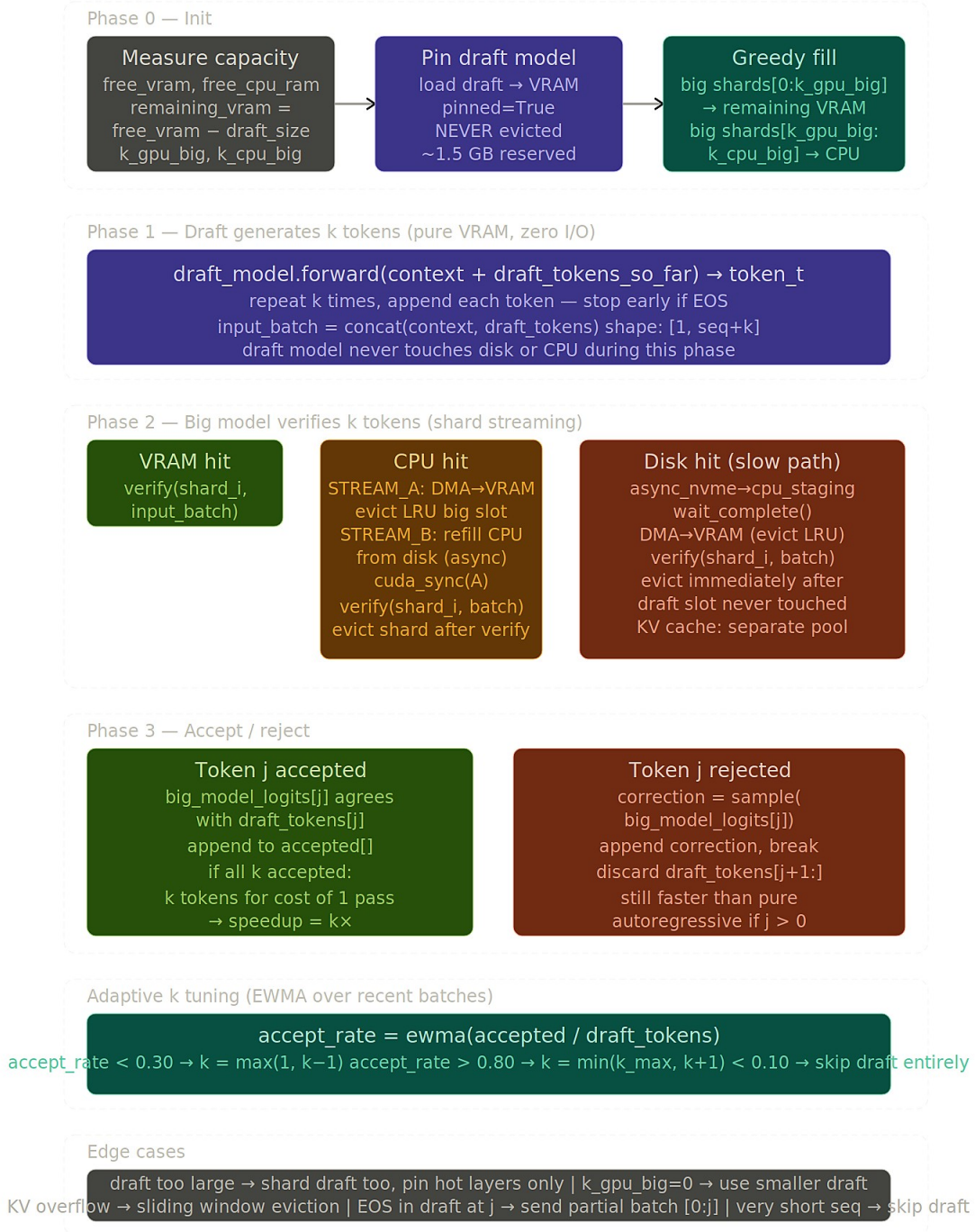


FIG. 7. Speculative decoding pipeline with shard streaming: a pinned draft model generates k candidate tokens with zero storage input/output; the large model then verifies all k candidates in a single shard-streaming sweep whose cost is independent of k ; tokens are accepted or rejected; and an adaptive controller tunes k from a rolling acceptance rate. Edge cases (oversized draft, zero accelerator memory, key-value overflow, early end-of-sequence) are handled explicitly.

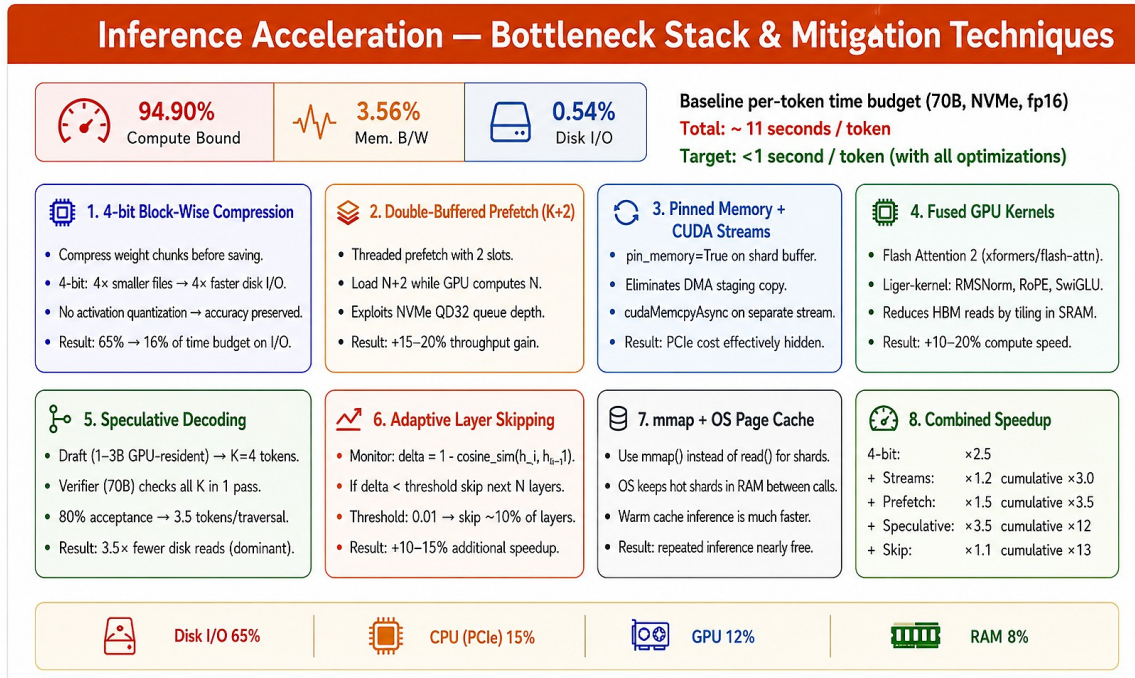


FIG. 8. Inference acceleration techniques and how they compose, including quantization, prefetching, pinned-memory transfer, fused kernels, speculative decoding, and adaptive layer handling (illustrative).

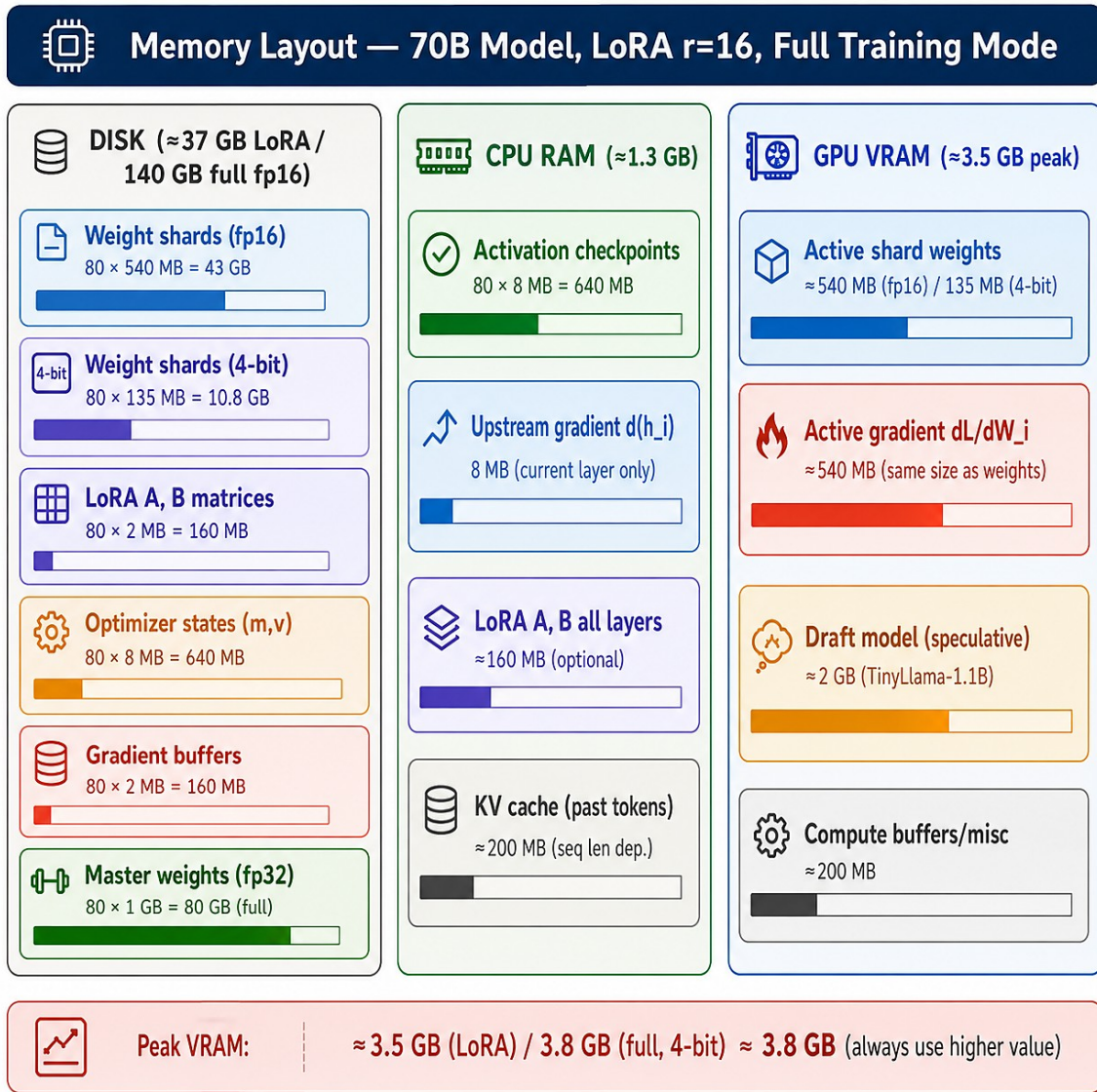


FIG. 9. Memory layout for a 70B model in low-rank-adaptor training mode: the contents and approximate footprint of each tier (storage, CPU RAM, accelerator memory) during full training.

Provide a brief summary of points of novelty in the Invention, as per the inventor.

- Adaptive Multi-Shard Engine (AMSE) — Per-Token Memory-Adaptive Layer Grouping.
 - After every generated token, the system measures available accelerator memory and computes k = the number of transformer layers that currently fit, then groups k layers into a single storage read. This reduces the number of storage read operations per forward pass from one per layer to one per group.
 - k is recomputed continuously and decreases automatically as the key-value cache grows during long generations — unlike fixed-policy systems (AirLLM loads one layer at a time; PIPO fixes its pipeline once at start-up).

- The same mechanism scales across hardware: more available memory yields larger k and fewer reads; zero accelerator memory yields CPU-only execution.
2. Single-Device On-Device Training with a Disk-Resident Optimizer.
 - All training state for each layer — quantized weights, full-precision master weights, optimizer moments, and accumulated gradients — is stored as an independent co-located file bundle on storage. The optimizer updates one layer at a time, so peak memory is bounded to a small constant independent of total model size.
 - Training requires a single device with no minimum accelerator memory beyond activation memory, and no multi-GPU collective communication — unlike QLoRA (full model in GPU) and ZeRO-style systems (multi-GPU servers).
 3. Reverse-Order Shard-Streaming Backward Pass with a Provable Memory Ceiling.
 - During training, weight shards are streamed from storage in reverse layer order while activation checkpoints saved during the forward pass are reloaded from CPU RAM. At any moment only the current shard group and two activation tensors occupy accelerator memory, giving a mathematically provable upper bound on memory use.
 - No prior training system performs a reverse-order shard-streaming backward pass.
 4. Training-Free Dynamic Speculative Decoding for Storage-Streamed Inference.
 - A small draft model proposes K candidate tokens; the large model verifies all K in a single storage-streaming pass, whose input/output cost is essentially independent of K . A controller raises or lowers K based on a rolling window of recently observed acceptance rates.
 - The controller requires no additional trained component and no gradient computation, distinguishing it from systems that fix K (Qualcomm US20250245430A1, SpecExec) or train a prediction head (SpecDec++).
 5. Draft Key-Value Cache Persistence (Linear Draft Cost).
 - The draft model retains its key-value cache between speculative steps, processing only newly generated tokens, which reduces total draft computation from quadratic to linear in the number of generated tokens.
 6. Heterogeneous, Per-Token CPU/Accelerator Placement.
 - Each shard group is assigned, independently and after each generated token, to the accelerator when memory permits or to the CPU when it does not, with intermediate results transferred between tiers. The system degrades gracefully to CPU-only execution mid-generation rather than failing when accelerator memory is exhausted.
 - Placement is decided dynamically per group per token, unlike static-partitioning systems (Fiddler routes expert weights statically; SpecOffload fixes an attention/feed-forward split).
 7. Framework-Agnostic Architecture Registry (30+ Model Families).
 - A registry maps each model family's identifier to its layer, embedding, and head locations, enabling shard splitting of 30+ transformer families — including LLaMA, Qwen, Mistral, Gemma, Phi, Falcon, DeepSeek, and mixture-of-experts variants — directly from standard checkpoint files without model-specific code.

8. Per-Shard Cryptographic Integrity Verification.

- Each weight shard carries a cryptographic checksum computed at creation and verified before use, detecting silent storage corruption before it can reach the accelerator — a safeguard no prior LLM execution framework provides.

9. Sparsity / Mixture-of-Experts Routing — Load Only Active Experts (Lever 1).

- For mixture-of-experts and sparsely-activated models, the system predicts which experts or feed-forward neurons a token will use and loads only those from storage — typically a small fraction of the feed-forward parameters — keeping frequently-used (“hot”) experts resident in accelerator memory and streaming rarely-used (“cold”) experts on demand, directly reducing the bytes loaded per token.

10. Deterministic Ring-Buffer Prefetch (Lever 1).

- Because the forward execution order of transformer layers is fixed and known in advance, the system replaces general cache-eviction heuristics with a deterministic, scheduled ring buffer and look-ahead prefetch, so that the storage “slow path” never fires in steady state — the next shard is always already in flight before it is needed.

11. Cross-Request Shard Amortisation — Batching (Lever 2).

- Because loading a shard from storage is the dominant cost, the system loads each shard once and processes many queued sequences or requests through that resident shard before evicting it, amortising a single shard load across multiple sequences and converting a bandwidth-bound workload into a compute-bound one.

12. Tree-Structured Speculative Verification (Lever 2).

- Exploiting the property that one storage-streaming verification sweep costs essentially the same regardless of how many candidate tokens are checked, the system drafts a tree of candidate continuations and verifies all branches in a single sweep, keeping the longest accepted path — so wider speculation is nearly free, distinct from linear single-path speculative decoding.

13. Idle-Tier Overlap Prefetch (Lever 2).

- During the draft phase, when only the small draft model runs and the storage and interconnect links are idle, the system uses that otherwise-wasted bandwidth to prefetch the large model’s first verification shards, so no tier is ever left idle.

14. Tier-Placement / Pre-Staging Resource Orchestrator.

- A scheduler component decides what data to place in which tier and when — hot weights and the pinned draft model in accelerator memory, warm data and optimizer state staged in CPU pinned RAM, and cold weights and key-value overflow on storage — moving data to the correct tier ahead of need based on the predicted execution order and current memory availability.

Fundamental Innovation Statement

The invention transforms execution of large language models from a process bounded by accelerator memory into one bounded only by available storage, making both inference and training of very large models possible on essentially any device — from a 4 GB consumer GPU down to a CPU-only machine — while adapting automatically to whatever hardware is present.

Indicate possible uses/applications for the invention. Does it enhance the usefulness of an existing technology? Are there other uses that might be realized in future?

Core Immediate Applications

- **On-device large-model inference:** Running 70B-class models locally on consumer laptops and entry-level GPUs that otherwise could not load them, preserving data privacy by keeping all processing on-device.
- **On-device fine-tuning:** Fine-tuning 70B-class models on a single consumer GPU with as little as 4 GB of memory, removing the need for multi-GPU servers for many customization tasks.
- **Universal hardware support:** A single engine that runs the same model on a 4 GB laptop GPU, a high-end workstation card, a server accelerator, or a CPU-only edge box — adapting automatically without reconfiguration.
- **Edge and offline AI:** Powering offline assistants and edge intelligence in environments with limited or no network connectivity.

Enhancement of Existing Technology

- The engine can serve as the execution back-end selected by a higher-level deployment system (such as QpiAI's hardware-aware deployment platform) whenever a chosen model is too large for any conventional engine on the target hardware.
- It extends parameter-efficient fine-tuning techniques (LoRA/QLoRA) to model sizes and devices previously out of reach.

Future Applications

- Extension to additional accelerator types (NPUs, TPUs, and future AI chips) through the hardware-agnostic memory-probing interface.
- Federated and distributed fine-tuning across many memory-constrained edge devices.
- Always-on personal and embedded assistants on phones, wearables, and appliances.

PRIOR DISCLOSURE INFORMATION

Have any patent applications been filed by you, which are closely related to the Invention? If yes, please provide publication number.

Yes — a related but distinct QpiAI application, No. 144678-0132 (“Method and System for Hardware-Aware Deployment and Optimization of Large Language Models across Heterogeneous Computing Architectures”), addresses the complementary problem of deciding and orchestrating which model, quantization, and existing inference engine to deploy on given hardware. Relationship: the prior application is a decision-and-orchestration system; the present invention is the distinct execution engine that physically runs a model larger than memory. The prior system may select the present engine as its deployment target. The present invention uses quantization only as a component that reduces per-layer shard size and does not claim any quantization algorithm or any method of selecting among quantization strategies (the subject of the prior application). There is no claim overlap.

Has any product been designed/launched which incorporates the invention? If yes, please provide details to determine novelty.

No.

Are you planning any disclosure of the Invention/launch of product implementing the invention in the near future (within next 6 months)? If yes, please provide approximate dates and locations.

Yes. The inventors may publish a research paper or preprint describing part of the subject matter of this invention within approximately the next twelve months. The specific date, venue, and scope are not yet finalized. Any such publication will occur only after this application has been filed. The applicant intends to file this application before any public disclosure or publication, and requests that counsel advise on filing timing accordingly.

Patent Claims

(Drafted in plain form for counsel review; counsel may restructure, split, or renumber as appropriate. Independent claims are marked.)

Note for counsel: with the addition of the sparsity, batching, tree-speculation, deterministic-prefetch, and resource-orchestration mechanisms, the disclosure now spans a family of related streaming optimisations. Counsel may wish to consider whether to pursue these as one application with multiple independent claims or to divide them into separate applications.

Claim 1 (Independent)

A system for executing inference of a large language model on a single computing device without distributed data-parallel collective communication, the system comprising: a hardware profiler that measures, after each generated output token, available accelerator memory and available central-processor memory, wherein the accelerator memory comprises one of graphics-processor video memory, neural-processing-unit memory, or zero in the case of a device having no dedicated accelerator; a shard-width calculator that computes a parallel layer count k as the number of transformer layers that fit within the available accelerator memory after subtracting a safety margin, wherein k is recomputed after each generated output token such that k decreases as a key-value cache consumes accelerator memory during generation; a strategy selector that assigns single-shard execution when k equals one, multi-shard execution when k is greater than one and less than the total layer count, full-model execution when k is at least the total layer count, and central-processor-only execution when k equals zero; a shard scheduler that loads k transformer-layer weight tensors simultaneously into the accelerator memory; and an execution engine that processes input through the k resident layers, evicts them, and loads the next k layers until all transformer layers have been processed.

Claim 2 (Dependent on Claim 1)

The system of Claim 1, wherein the shard scheduler concurrently performs computation of the current shard group on the accelerator, transfer of the next shard group from central-processor memory to the accelerator, and reading of a subsequent shard group from non-volatile storage into central-processor memory, such that the effective time per shard group approaches the maximum of the three operation times rather than their sum.

Claim 3 (Dependent on Claim 1)

The system of Claim 1, wherein the shard-width calculator recomputes k by measuring the accelerator memory consumed by the key-value cache at the current token position, such that the system transitions automatically from multi-shard to single-shard execution without interrupting generation.

Claim 4 (Dependent on Claim 1)

The system of Claim 1, wherein, when k equals zero, the shard scheduler loads transformer-layer weight tensors into central-processor memory and the execution engine performs all computation on central-processor compute units, the system transitioning automatically from accelerator-assisted to central-processor-only execution during a generation session as the key-value cache consumes the available accelerator memory, without restarting or reloading the model.

Claim 5 (Dependent on Claim 1)

The system of Claim 1, further comprising a placement engine that assigns each shard group, independently after each generated token, to accelerator computation when accelerator memory headroom exceeds a threshold or to central-processor computation otherwise, transferring intermediate results between tiers, wherein the placement decision is made dynamically per shard group per generated token.

Claim 6 (Dependent on Claim 1)

The system of Claim 1, further comprising a model-architecture registry that maps a model-family identifier to the locations of that family's transformer layers, embedding, and output head, for at least thirty transformer families including dense decoder-only families and mixture-of-experts families, enabling shard splitting directly from standard checkpoint files without model-family-specific program code.

Claim 7 (Independent)

A system for reducing the number of non-volatile storage read operations during execution of a large language model, the system comprising: a shard-width calculator that computes a parallel layer count k as the number of transformer layers that fit within available accelerator memory after subtracting a safety margin, k being at least one; a shard grouper that serialises k consecutive transformer-layer weight tensors as a single logical shard unit and issues a single sequential storage read per shard unit, thereby reducing the number of storage read operations per forward pass from one per transformer layer to one per shard unit; and an execution engine that processes all k layers resident in accelerator memory before evicting the shard unit and loading the next, wherein the parallel layer count k increases with available accelerator memory so that a device with greater available memory issues proportionally fewer read operations.

Claim 8 (Dependent on Claim 7)

The system of Claim 7, wherein the shard-width calculator recomputes k after each generated output token to reflect growth of a key-value cache, and the shard grouper adjusts the boundary between consecutive shard units accordingly throughout the generation session.

Claim 9 (Independent)

A system for training a large language model on a single computing device without distributed data-parallel collective communication, the system comprising: a per-layer file store on non-volatile storage holding, for each transformer layer, a co-located bundle of independent files comprising a quantized weight file, a full-precision master weight file, an optimizer-state file, and an accumulated-gradient file; a forward engine that loads transformer-layer weight groups in sequence and writes each hidden-state tensor to central-processor memory at each group boundary; a backward engine that loads weight groups in reverse order, reloads the stored hidden-state tensors from central-processor memory, computes gradients, and accumulates them to the per-layer gradient files; and an optimizer that applies updates to the full-precision master weights one layer at a time, re-quantizes, writes the updated files back to storage, and deletes the gradient file, wherein peak central-processor memory during the

optimizer step is bounded to a constant multiple of a single layer's weight size independent of total model size, and no minimum accelerator memory is required beyond activation memory.

Claim 10 (Independent)

A system for computing gradients of a large language model on a single computing device, the system comprising: a central-processor-memory activation store holding hidden-state tensors written at each transformer-layer-group boundary during a forward pass; and a backward engine that iterates the transformer-layer groups in strictly reverse order and, for each group, loads the group's weight tensors from non-volatile storage, reloads the stored hidden-state tensors from the activation store, reconstructs a local automatic-differentiation graph, computes weight gradients and an upstream gradient, accumulates the weight gradients to per-layer storage, and evicts the group's weight tensors before proceeding to the preceding group, such that at any moment only the current group and two activation tensors occupy accelerator memory, establishing a provable upper bound on accelerator-memory use.

Claim 11 (Dependent on Claim 9)

The system of Claim 9, further comprising a low-rank adapter manager that injects trainable low-rank adapter matrices into each layer while keeping the base weights frozen, wherein gradients are computed only for the adapter matrices and the adapter gradients for all layers are held simultaneously in central-processor memory.

Claim 12 (Dependent on Claims 9 and 11)

The system of Claims 9 and 11, wherein all optimizer state for the adapter matrices of all layers is held simultaneously in central-processor memory throughout training, such that the optimizer step for adapter fine-tuning requires no non-volatile storage read or write operations, in contrast to full-parameter fine-tuning in which the optimizer step reads and writes per-layer optimizer-state files.

Claim 13 (Dependent on Claims 9 and 10)

The system of Claims 9 and 10, wherein the backward engine reloads frozen base-weight groups from storage in reverse order to propagate the upstream gradient through the full layer, while computing parameter gradients only for the adapter matrices, such that the storage read pattern of the backward pass mirrors that of the forward pass regardless of whether full-parameter or adapter-only fine-tuning is performed.

Claim 14 (Dependent on Claim 9)

The system of Claim 9, further comprising a checkpoint manager that saves, at configurable intervals, the adapter weights, optimizer-state snapshots, an index of the last-processed training sample, and training configuration, and that restores all such state to resume training from the exact step and data position after an interruption.

Claim 15 (Independent)

A system for speculative inference of a large language model whose weights are streamed from non-volatile storage, the system comprising: a draft model resident in accelerator memory that generates K candidate tokens; a verifier that executes a single storage-streaming forward pass of the large model

over the sequence extended by the K candidate tokens, verifying all K tokens at a per-pass storage cost that is substantially independent of K; an accept-reject sampler that accepts or rejects each candidate token by comparing large-model and draft-model probabilities; and a draft-length controller that maintains a rolling window of observed acceptance rates and increases or decreases K according to the rolling-mean acceptance rate, wherein the controller requires no additional trained component and no gradient computation.

Claim 16 (Dependent on Claim 15)

The system of Claim 15, wherein the draft model retains its key-value cache between consecutive speculative steps and processes only newly generated tokens at each step, reducing total draft computation from quadratic to linear in the number of generated tokens.

Claim 17 (Independent)

A system for ensuring integrity of transformer-layer weight shard files, the system comprising: a serializer that computes a cryptographic hash of each shard file immediately after creation and writes the hash to a companion file; a deserializer that recomputes and compares the hash before use and raises an error identifying the shard if the hashes differ; and a verifier that reports verified, missing, and corrupted shards before execution begins.

Claim 18 (Independent — Sparsity / Expert Streaming)

A system for executing a sparsely-activated large language model, the system comprising: an expert-prediction component that determines, for a current input token, a subset of experts or feed-forward neurons that will be activated; a residency manager that maintains frequently-used experts in accelerator memory and stages remaining experts on non-volatile storage; and a loader that, for each token, loads from storage only those experts in the determined subset that are not already resident, such that the quantity of expert parameters loaded per token is a fraction of the total feed-forward parameters of the model.

Claim 19 (Dependent on Claim 1 — Deterministic Prefetch)

The system of Claim 1, wherein, because the forward execution order of the transformer layers is fixed and known in advance, the shard scheduler prefetches shards according to a deterministic schedule using a ring buffer and a configurable look-ahead distance, rather than according to a cache-eviction heuristic, such that, in steady-state operation, each shard is present in accelerator memory or central-processor memory before it is required and the non-volatile storage read latency is hidden behind computation.

Claim 20 (Independent — Cross-Request Shard Amortisation)

A system for executing a large language model whose weights are streamed from non-volatile storage for a plurality of concurrent inference requests, the system comprising: a request queue holding a plurality of input sequences; a shard scheduler that loads a transformer-layer shard into accelerator memory once and, while that shard is resident, processes a plurality of the queued input sequences through it before evicting the shard; and an execution engine that repeats this for successive shards, such that the cost of loading each shard from storage is amortised across the plurality of sequences and

the workload transitions from bandwidth-bound toward compute-bound as the number of concurrent sequences increases.

Claim 21 (Dependent on Claim 15 — Tree-Structured Speculative Verification)

The system of Claim 15, wherein the draft model generates a tree of candidate token continuations comprising a plurality of branches, and the verifier verifies all branches of the tree in a single storage-streaming sweep of the large model whose storage cost is substantially independent of the number of candidates verified, the system selecting the longest branch whose tokens are all accepted, thereby accepting multiple tokens per sweep while loading the large-model weights only once.

Claim 22 (Dependent on Claim 15 — Idle-Tier Overlap Prefetch)

The system of Claim 15, wherein, during generation of the candidate tokens by the draft model — a phase during which the draft model resides in accelerator memory and the non-volatile storage and interconnect links are otherwise idle — the system prefetches one or more of the large model's verification shards from non-volatile storage into central-processor memory or accelerator memory, such that the subsequent verification sweep begins with its first shards already staged.

Claim 23 (Dependent on Claim 1 — Tier-Placement Resource Orchestrator)

The system of Claim 1, further comprising a placement orchestrator that assigns each data item to one of the three memory tiers ahead of its time of use based on the known forward execution order and the current available memory of each tier, the orchestrator maintaining frequently-used weights and a pinned draft model in accelerator memory, staging warm weights and optimizer state in central-processor pinned memory, and retaining cold weights and key-value-cache overflow on non-volatile storage, and promoting or demoting data between tiers as predicted usage changes.

Claim 24 (Independent — Method)

A computer-implemented method for executing a large language model on a single computing device comprising non-volatile storage, central-processor memory, and an accelerator, the method comprising: after each generated output token, measuring available accelerator memory; computing a parallel layer count k as the number of transformer layers that fit within the available accelerator memory after a safety margin; selecting single-shard, multi-shard, full-model, or central-processor-only execution based on k ; loading k transformer-layer weight tensors simultaneously from storage; executing a forward pass through the k resident layers while prefetching the next group; and repeating until all layers have been processed.

Claim 25 (Dependent on Claim 24)

The method of Claim 24, further comprising generating K candidate tokens with a resident draft model retaining its key-value cache between steps, verifying all K in a single storage-streaming pass, accepting or rejecting each candidate, and adjusting K from a rolling window of observed acceptance rates without any trained component.

Claim 26 (Dependent on Claim 24)

The method of Claim 24, wherein, when k equals zero during a generation session, the method continues generating tokens by loading transformer-layer weight tensors into central-processor memory and computing on central-processor compute units, without restarting or reloading the model.

Claim 27 (Independent — Method)

A computer-implemented method for training a large language model on a single computing device, the method comprising: storing, for each transformer layer, a full-precision master weight file, an optimizer-state file, and an accumulated-gradient file on non-volatile storage; executing a forward pass by loading transformer-layer weight groups in sequence and writing each hidden-state tensor to central-processor memory; executing a backward pass by loading weight groups in reverse order, reloading the stored hidden-state tensors, computing gradients, and accumulating them to the per-layer gradient files; and executing an optimizer step by updating the master weights one layer at a time, re-quantizing, writing the updated files back to storage, and deleting the gradient file.

Claim 28 (Dependent on Claim 27)

The method of Claim 27, further comprising injecting trainable low-rank adapter matrices with the base weights frozen, computing gradients only for the adapter matrices, holding all adapter gradients in central-processor memory simultaneously, and performing the adapter optimizer step without non-volatile storage input or output.

Claim 29 (Independent — Computer-Readable Medium)

A non-transitory computer-readable medium storing instructions that, when executed by one or more processors of a single computing device, cause the device to perform the method of any one of Claims 24 through 28.